Lecture VIII - Pandas and gAI

Programming with Python

Dr. Tobias Vlćek



Checkpoint 4

Sessions VI-VII. The first 40 minutes are the checkpoint. It starts now.

- **Individual work**: no neighbors, no messaging
- **Any gAI tool is allowed**: the graded skill is to **VERIFY** its output
- The **link is on Moodle**. Open it and start
- 12 short tasks: Tobi's chatbot wrote the cells. You keep the right ones, fix the wrong ones.
- It sweeps **Sessions VI-VII**: modules, `random` and seeds, NumPy arrays and masks

When you're done

Cmd/Ctrl+S, then menu → *Download* → *Download Python code* → upload the `.py` to the **"Checkpoint 4"** assignment on Moodle, before the 40 minutes are up. **No retakes**: one session.

> **Note**
> **No live checks this time.** Nothing in the notebook tells you whether you're right: checking is the job. The grading runs on my side. Everything in it was rehearsed in the labs.

EP 8: The Data Room

Show me the data

This morning rival chain MunchCorp bragged in a press release about their "data-driven growth." The evidence attached: exactly **one pie chart.** The investor slid it across the table: *"Yours will be better. Show me the data."*

Tobi didn't wait. He'd pasted the question into a chatbot, proudly turned the laptop around, and hit run:

```
AttributeError: 'DataFrame' object has no attribute 'overview'
```

The gAI sounded certain, but the method it called **does not exist.** Today starts with the tool that answers the investor for real, **pandas**, and then comes back to that difference.

The DataFrame

What pandas adds over NumPy

Last session, a NumPy array turned a thousand numbers into one fast object, but every value had to share **one type**, and columns had no names.

- A **DataFrame** is a table: named columns, and each column can be a different type
- Text zones next to float euros next to integer counts, all in one object
- It's the spreadsheet the investor handed you, loaded into Python

The convention everyone uses: `import pandas as pd`.

A DataFrame from a dictionary

Keys become **column names**; each list becomes a **column**; one order per row, mixed types together:

```
import pandas as pd
df = pd.DataFrame({
    "order_id":  [101, 102, 103, 104, 105],
    "zone":      ["Nord", "Sued", "Nord", "Hafen", "Sued"],
    "items":     [2, 1, 3, 1, 4],
    "total_eur": [18.50, 7.20, 24.00, 6.80, 31.40],
})
print(df)
```

```
   order_id   zone  items  total_eur
0       101   Nord      2       18.5
1       102   Sued      1        7.2
2       103   Nord      3       24.0
3       104  Hafen      1        6.8
4       105   Sued      4       31.4
```

A DataFrame from a file

> **Note**
> Nobody types two weeks of orders by hand: today's lab loads a real **CSV** (a table saved as plain text, one row per line) in one line, `orders = pd.read_csv("orders_messy.csv")`. The file sits in the same folder as your script, so its name is all the path you need.

First look: `.head()` and `.info()`

Glance at a table first. `.head()` shows the top rows; `.info()` is your first sanity check: right number of rows? Any surprising type?

```
print(df.head(3))     # first 3 rows
df.info()             # columns, dtypes, non-null counts
```

```
   order_id  zone  items  total_eur
0       101  Nord      2       18.5
1       102  Sued      1        7.2
2       103  Nord      3       24.0
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5 entries, 0 to 4
Data columns (total 4 columns):
 #   Column     Non-Null Count  Dtype  
---  ------     --------------  -----  
 0   order_id   5 non-null      int64  
 1   zone       5 non-null      object 
 2   items      5 non-null      int64  
 3   total_eur  5 non-null      float64
dtypes: float64(1), int64(2), object(1)
memory usage: 292.0+ bytes
```

Numbers at a glance: `.describe()`

`.describe()` computes count, mean, min, max and quartiles for every **numeric** column at once:

```
print(df.describe())
```

```
         order_id    items  total_eur
count    5.000000  5.00000   5.000000
mean   103.000000  2.20000  17.580000
std      1.581139  1.30384  10.688873
min    101.000000  1.00000   6.800000
25%    102.000000  1.00000   7.200000
50%    103.000000  2.00000  18.500000
75%    104.000000  3.00000  24.000000
max    105.000000  4.00000  31.400000
```

One call, the whole summary: this is the `.overview()` Tobi's chatbot wished existed.

Selecting a column

Pick one column by name:

```
print(df["zone"])                  # one column (a Series)
```

```
0     Nord
1     Sued
```

```
2     Nord
3    Hafen
4     Sued
Name: zone, dtype: object
```

Filtering rows

Keep rows with a **boolean mask**, exactly the NumPy idea from last session:

```
print(df[df["zone"] == "Nord"])    # only the Nord rows, case-sensitive!
```

```
   order_id  zone  items  total_eur
0       101  Nord      2       18.5
2       103  Nord      3       24.0
```

`df["zone"] == "Nord"` builds a column of `True`/`False`; indexing with it keeps the `True` rows. `.loc` exists for label-based selection, but a plain mask covers today.

Predict: what comes back?

Tobi wants the orders over 20 EUR. He compares the column and prints the result:

```
big = df["total_eur"] > 20
print(big)
```

a) a column of `True`/`False`, one per row
b) only the rows whose total is over 20
c) a single `True`, since some order is over 20

Answer: a mask, not a table

```
big = df["total_eur"] > 20
print(big)               # the mask: a Series of booleans
print(df[big])           # the rows: build the mask, then apply it
```

```
0    False
1    False
2     True
3    False
4     True
Name: total_eur, dtype: bool
   order_id  zone  items  total_eur
2       103  Nord      3       24.0
4       105  Sued      4       31.4
```

a) a column of `True`/`False`. The comparison alone is the **mask**, a Series with one boolean per row. The rows only appear once you index with it.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_08_a/



Any questions? Let me know!

Adding a column, sorting

Compute a new column from existing ones, then sort. Work on a **copy** so the original stays intact:

```
priced = df.copy()
priced["eur_per_item"] = priced["total_eur"] / priced["items"]
print(priced.sort_values("total_eur", ascending=False))
```

```
   order_id   zone  items  total_eur  eur_per_item
4       105   Sued      4       31.4          7.85
2       103   Nord      3       24.0          8.00
0       101   Nord      2       18.5          9.25
1       102   Sued      1        7.2          7.20
3       104  Hafen      1        6.8          6.80
```

`sort_values` reorders rows by a column, biggest orders first here. The new column is just arithmetic on two others, computed for every row at once.

Predict: filter, wrong case

Tobi filters for the Nord zone, but types it **lowercase**. The data spells it `"Nord"`. What does this print?

```
print(df[df["zone"] == "nord"])
```

a) an empty table, no error
b) the Nord rows anyway: case is ignored
c) a `KeyError` for the missing zone

Answer: an empty table, no warning

```
print(df[df["zone"] == "nord"])          # nothing matches "nord"
print("rows:", len(df[df["zone"] == "nord"]))
```

```
Empty DataFrame
Columns: [order_id, zone, items, total_eur]
Index: []
rows: 0
```

a) an empty table. `"nord"` matches nothing, so the mask is all `False` and pandas hands back **zero rows**. No error…

This is the dangerous kind of bug: it **runs**, it just returns nothing. **pandas won't warn you**, which is exactly why we try to test filters on a case whose answer we know.

One more tool: `groupby`

The investor's real question is **per zone**: which area brings in the most?

- `groupby` splits the table by a category, then computes **once per group**
- `df.groupby("zone")["total_eur"].sum()` → one total for each zone

> **Note**
> That's the teaser: the lab needs more of `groupby` than this slide shows. It is a good one to **ask your agent** about. How to check what it hands you comes next.

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_08_b/



Any questions? Let me know!

gAI joins the team

Prompting: context + constraints

Two things turn a vague request into a useful answer:

- **Context**. Say what the data *is*: "I have a DataFrame `orders` with columns `zone` (text) and `total_eur` (float)."
- **Constraints**. Say exactly what you want *back*: "Give me the total `total_eur` for zone `Nord`, as a single number."

> *"DataFrame `orders`, text column `zone`, float column `total_eur`. Write one line that returns the total `total_eur` where `zone` is `Nord`."*

Vague in, vague out. Specific in, checkable out, then you **iterate**.

The verify workflow

Tobi's mistake wasn't *using* gAI. It was **shipping without checking.** Every line an gAI hands you should get three passes if it is critical:

1. **Read it**: do you understand what each line claims to do?
2. **Run it**: does it actually execute, or does it crash?
3. **Test it**: try it on a small case where **you already know the answer.**

Step 3 in action

Five orders with values you can add in your head, and the gAI's suggested line for the Nord average:

```
import pandas as pd

check = pd.DataFrame({
    "zone":      ["Nord", "Sued", "Nord", "Hafen", "Nord"],
    "total_eur": [10.0, 7.5, 20.0, 5.0, 30.0],
})
print(check[check["zone"] == "Nord"]["total_eur"].mean())


20.0
```

The Nord orders are 10, 20 and 30. You can average those in your head. Does the gAI's answer match? Then the line has earned trust on the real file. With gAI, verification is often the job now, not typing the code yourself.

Predict: it ran, so it's right?

Tobi asks the gAI for the **Nord average** on the same five orders. It hands back one line and calls it done. What does this print?

```
print(check["total_eur"].mean())
```

a) `20.0`
b) an `AttributeError`
c) `14.5`

Answer: it ran, and it's wrong

```
print(check["total_eur"].mean())                              # every zone
print(check[check["zone"] == "Nord"]["total_eur"].mean())     # Nord only


14.5
20.0
```

c) **`14.5`**: the line averages **all five** orders. Nobody filtered for Nord, and pandas has no way to know that was the question.

No traceback, a plausible number, and still wrong. "Code runs" only clears step 2. Step 3 is where the actual number get tested.

Predict: does `.summarize()` exist?

The gAI wrote this for Tobi. `orders` is a tiny two-row frame. What happens on the last line?

```
import pandas as pd
orders = pd.DataFrame({"zone": ["Nord", "Sued"], "total_eur": [12.0, 9.5]})
```

```
orders.summarize()
```

a) prints a summary table
b) raises an `AttributeError`
c) returns an empty DataFrame

Answer: the method never existed

```
import pandas as pd
orders = pd.DataFrame({"zone": ["Nord", "Sued"], "total_eur": [12.0, 9.5]})
orders.summarize()                    # the method the gAI invented
```

```
AttributeError: 'DataFrame' object has no attribute 'summarize'
[31mAttributeError[39m[31m: [39m 'DataFrame' object has no attribute
'summarize'
```

b) **`AttributeError`**: pandas has no `.summarize()`. The gAI invented it.

The real method is `.describe()`

```
print(orders.describe())              # the method that actually exists
```

```
       total_eur
count   2.000000
mean   10.750000
std     1.767767
min     9.500000
25%    10.125000
50%    10.750000
75%    11.375000
max    12.000000
```

A gAI that sounds sure is not the same as a function or method that exists.

Why did it invent a method?

Session VI gave you the basics, so here is why it sometimes fails.

- Training rewards a **plausible** continuation and never checks a **true** one: nothing in the loop verifies facts
- And it always writes *some* next token, confident or not: a choice gets picked, printed, and it all looks equally smooth
- `.summarize` is a *plausible* pandas name: short, lowercase, shaped like `.describe()`.

Fluent and wrong is difficult to fix. It is how gAI works due to its stochastic nature.

How it was trained

- **Pre-training**: predict the next token over massive text, where the **weights** inside get adjusted. Applying them afterwards (called *inference*) is much cheaper
- **Fine-tuning**: curated examples teach it to follow instructions
- **RLHF** (reinforcement learning from human feedback): humans rank answers and it learns to prefer the ranked-higher ones
- Its knowledge has an end date, the **knowledge cutoff**: after it, it guesses fluently or uses web search. And it inherits the **biases** of the text it read!

Every stage rewards *sounding* right. Verifacation is your job (at least for now), and step 3 of the verify workflow is how.

Disclosure: say what the AI did

In your project (Part III) you will have to list each AI use in one line. Labs and checkpoints need no note. This is a good habit to get into for your own work. . . .

Although it sounds strange, it protects **you**: it separates the work you understand from the work you pasted, so when a reviewer (or the investor or a different professor) asks "how does this work?", you're never caught claiming something you can't explain.

When NOT to reach for AI

Sometimes the fastest path is a different one:

- A red traceback? **You've read those since Episode 5.** The last line names the error and points at the file, often faster than describing it to a chatbot.
- A one-line filter you've written ten times? Just write it.

> **Tip**
> Use gAI to draft the unfamiliar and to explain the confusing, not to skip thinking you can do yourself (at least while you are in university).

Your turn

Open the exercise

Scan the QR or type the link:

python.tobiasvlcek.com/notebooks/ex_08_c/



Any questions? Let me know!

To the Lab

After the break: the lab

- The lab is a **script** now: download `lab_08_dataroom.py` and `orders_messy.csv` from Episode 8: The Data Room and move them into `python-labs`
- In Zed's terminal: `uv run python lab_08_dataroom.py`. It checks every task and stops at the first one without a valid answer
- The USB stick holds Tobi's raw export. Each task gives you a symptom ("Hafen is missing orders"); you find the cause
- **One problem in the file goes unmentioned**; no check finds it

> ! **Important**
> **Show me the last block before you leave**, for feedback: the script prints `SHOW THIS IN CLASS`, and I compare your numbers with mine.

Your agent, in Zed

Open the agent panel (the sparkle icon, bottom right), pick **Mistral Vibe**, and ask for **one task at a time**:

> *In `lab_08_dataroom.py`, fill in `revenue_by_zone` in Task 4: a dict, zone name to euros, from the DataFrame `clean`. Change nothing else.*

1. **Read it**: Zed shows the changes as a *diff*, old lines red, new lines green. Read it **before** you accept
2. **Run it**, then **test it** on a case where you know the answer

More on how gAI works and how you can use it even better in Session X. Today it is a tool you supervise.

Wrap-up

Three things to remember

1. **A DataFrame is a named, mixed-type table.** `pd.DataFrame` from a dict, `pd.read_csv` from a file; `.head()`, `.info()`, `.describe()` to look before you leap.
2. **Select, filter, add, group.** `df["col"]`, a boolean mask (`df[df["zone"] == "Nord"]`, case-sensitive!), a new column on a `.copy()`, `groupby` per category. pandas won't warn; you verify.
3. **AI drafts, you verify.** Give it context and constraints, then read it → run it → test it on a case you know.

> **Note**
> **Next episode: charts the investor can't argue with.** Numbers convince the careful and a good plot convinces the room. We turn the data room into pictures.

Literature

Books to start with

> **Note**
> Working with AI this session? Revisit the AI Tools page: context-and-constraints prompting, the verify workflow, and the disclosure rule for your project. For pandas itself, the official 10 minutes to pandas guide is the friendliest next step.

For more, see the literature list of this course.

Bibliography